

Table Per Type (TPT) Inheritance in Entity Framework Core

In this article, I will discuss Table Per Type (TPT) Inheritance, how to implement it in Entity Framework Core (EF Core), and provide a step-by-step real-time example. Please read our previous article discussing [Table Per Hierarchy \(TPH\) Inheritance in EF Core](#) with Examples. Entity Framework Core (EF Core) supports different types of inheritance mapping strategies to represent inheritance hierarchies in the database, one of which is Table Per Type (TPT).

Table Per Type (TPT) Inheritance in Entity Framework Core

Table Per Type (TPT), also known as Class Table Inheritance, is an inheritance mapping strategy where each class in an inheritance hierarchy is stored in its own database table. The base class properties are stored in one table, and each derived class has its own table containing only the properties specific to that class.

In this approach, the derived class tables store only the additional properties that aren't present in the base class. EF Core joins the base and derived tables when querying for derived entities, which provides a normalized database schema.

This approach avoids null columns since each table only contains the properties relevant to that entity type. However, it can introduce performance overhead due to the necessity of joining multiple tables to retrieve data for derived types.

Key Features of Table Per Type (TPT) Inheritance:

- **Separate Tables:** Each entity in the inheritance hierarchy is stored in its own database table.
- **Foreign Key Relationships:** Derived class tables have a primary key that is also a foreign key referencing the base table's primary key, forming a one-to-one relationship.
- **No Null Columns:** Since each table contains only the properties for that class, there are no null columns due to the normalized schema compared to TPH.

How to Implement Table Per Type (TPT) Inheritance in Entity Framework Core

We need to follow the below steps to implement TPT inheritance in Entity Framework Core:

- **Create Base Class and Derived Classes:** Define the base class for shared properties and derived classes for specific properties.
- **Configure TPT using Fluent API:** Use the Fluent API to specify that each entity in the hierarchy should map to its own table using the `.ToTable()` method.

Real-time Example: Implementing TPT Inheritance in a Content Management System (CMS)

Consider a scenario where we are developing a Content Management System (CMS) that handles different types of content, such as Articles, Videos, and Images. Each content type shares common attributes but has specific properties unique to its category. We will implement TPT inheritance to model this hierarchy in EF Core. Let us proceed and define the Base and Derived Entities:

Base Entity: Content

Create a new file named **Content.cs** within the Entities folder and add the following code. The Content class contains properties common to all content types and serves as the base class for all content types.

```
namespace EFCoreCodeFirstDemo.Entities
{
    // Base Class representing general content
```

```

public abstract class Content
{
    public int ContentId { get; set; } // Primary Key
    public string Title { get; set; } // Title of the content
    public string Author { get; set; } // Author of the content
    public DateTime PublishedDate { get; set; } // Date when the content
was published
    public ContentType ContentType { get; set; } // Type of the content
(e.g., Article, Video, Image)
    public ContentStatus Status { get; set; } // Status of the content
(Draft, Published, Archived)
    // Navigation Properties
    public virtual ICollection<Comment> Comments { get; set; } // Comments
related to the content
}
// Enum for Content Status
public enum ContentStatus
{
    Draft,
    Published,
    Archived
}
// Enum for Content Type
public enum ContentType
{
    Article,
    Video,
    Image
}
}

```

Comment Entity

We will define a Comment entity to represent comments on content items. Create a new file named **Comment.cs** within the Entities folder and add the following code.

```

namespace EFCoreCodeFirstDemo.Entities
{
    // Entity representing a Comment on Content
    public class Comment
    {
        public int CommentId { get; set; } // Primary Key
        public string AuthorName { get; set; } // Name of the commenter
        public string Text { get; set; } // Content of the comment
        public DateTime CommentedDate { get; set; } // Date when the comment
was made
        public int ContentId { get; set; } // Foreign Key to Content
        public virtual Content Content { get; set; } // Navigation property
to Content
    }
}

```

```
}
```

Derived Entity: Article

Create a new file named **Article.cs** within the Entities folder and add the following code. The Article class inherits from Content and includes properties specific to articles.

```
namespace EFCoreCodeFirstDemo.Entities
{
    // Derived Class representing an Article
    public class Article : Content
    {
        public string Content { get; set; } // Full content of the article
        public string Summary { get; set; } // Brief summary of the article
        public int? ReadingTime { get; set; } // Estimated reading time in
minutes
        public string? FeaturedImage { get; set; } // URL of the featured
image
        public DateTime? LastEditedDate { get; set; } // Date when the
article was last edited
        public string? MetaKeywords { get; set; } // SEO keywords
        public string? MetaDescription { get; set; } // SEO meta description
        public string? MetaTitle { get; set; } // SEO Title
    }
}
```

Derived Entity: Video

Create a new file named **Video.cs** within the Entities folder and add the following code. The Video class inherits from Content and includes properties specific to videos.

```
namespace EFCoreCodeFirstDemo.Entities
{
    // Derived Class representing a Video
    public class Video : Content
    {
        public string VideoUrl { get; set; } // URL of the video
        public int Duration { get; set; } // Duration of the video in seconds
        public string ThumbnailUrl { get; set; } // URL of the thumbnail
image
        public string Resolution { get; set; } // Video resolution (e.g.,
1080p)
        public bool HasSubtitles { get; set; } // Indicates if subtitles are
available
        public string? Subtitles { get; set; } // URL of the subtitles file
        public string? MetaKeywords { get; set; } // SEO keywords
        public string? MetaDescription { get; set; } // SEO meta description
    }
}
```

Derived Entity: Image

Create a new file named **Image.cs** within the Entities folder and add the following code. The Image class inherits from Content and includes properties specific to images.

```
namespace EFCoreCodeFirstDemo.Entities
{
    // Derived Class representing an Image
    public class Image : Content
    {
        public string ImageUrl { get; set; } // URL of the image
        public string? Caption { get; set; } // Caption for the image
        public string AltText { get; set; } // Alternative text for
accessibility
        public string Dimensions { get; set; } // Dimensions of the image
(e.g., 1920x1080)
        public string? Photographer { get; set; } // Name of the photographer
    }
}
```

Configure the DbContext

Modify the EFCoreDbContext class as follows. This class represents the session with the database, allowing us to query and save instances of our entities. It includes the configuration necessary for Entity Framework Core to map entities to the database using TPT inheritance.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Configuring the connection string to the SQL Server database
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=CMSDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        // Configures the model and mappings between entities and database
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // Map each class in the hierarchy to its own table
            modelBuilder.Entity<Content>().ToTable("Contents"); // Base table
properties
            modelBuilder.Entity<Article>().ToTable("Articles"); // Table for Article
            modelBuilder.Entity<Video>().ToTable("Videos"); // Table for Video
            modelBuilder.Entity<Image>().ToTable("Images"); // Table for Image
            // Configure enums to be stored as strings
            // For ContentType enum
            modelBuilder.Entity<Content>()
                .Property(c => c.ContentType)
                .HasConversion<string>()
```

```

        .IsRequired();    // Optional: Specify if the property is required
    // For ContentStatus enum
    modelBuilder.Entity<Content>()
        .Property(c => c.Status)
        .HasConversion<string>()
        .IsRequired();    // Optional: Specify if the property is required
    }
    // DbSetes representing each table in the database
    public DbSet<Content> Contents { get; set; }
    public DbSet<Article> Articles { get; set; }
    public DbSet<Video> Videos { get; set; }
    public DbSet<Image> Images { get; set; }
    public DbSet<Comment> Comments { get; set; }
}
}

```

Generate Migration and Update Database

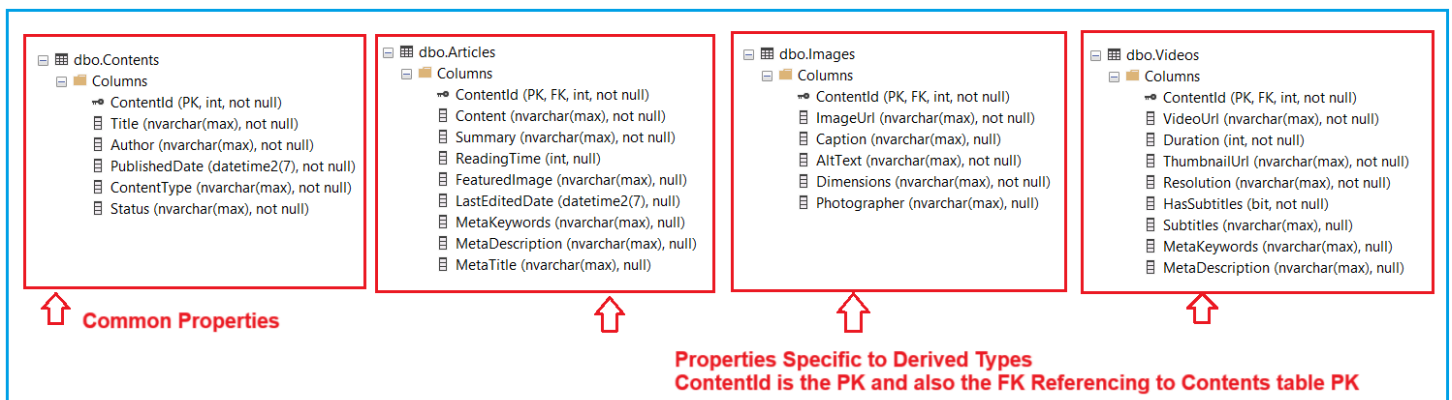
With the above changes, open the Package Manager Console and Execute the Add-Migration and Update-Database commands as follows.

```

Package Manager Console
Package source: All Default project: EFCoreCodeFirstDemo
PM> Add-Migration Mig1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> Update-database
Build started...
Build succeeded.
Applying migration '20240923141805_Mig1'.
Done.

```

Once you execute the above code, it should have created the database with the required tables, as shown in the image below. The Contents table contains the common properties, and other tables contain columns specific to the derived types.



Example of Insert Operations Using TPT Inheritance in EF Core

When adding entities of Article, Video, or Image, Entity Framework Core inserts the common properties (like ContentId, Title, Author) into the base Contents table and the type-specific properties into the corresponding derived table (Articles, Videos, or Images). EF Core handles the separation of the common and specific properties based on the inheritance structure and saves them across multiple tables.

For a better understanding, please modify the Program class as follows. The following example shows how to insert data into entities (Content, Article, Video, and Image) using Table Per Type (TPT) inheritance in Entity Framework Core.

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            using (var context = new EFCoreDbContext())
            {
                // Create and seed Article content
                var article = new Article
                {
                    Title = "Understanding EF Core TPT Inheritance",
                    Author = "Pranaya Rout",
                    PublishedDate = DateTime.Now,
                    ContentType = ContentType.Article,
                    Status = ContentStatus.Published,
                    Content = "This is a comprehensive guide on implementing  
TPT Inheritance...",
                    Summary = "EF Core TPT Inheritance",
                    ReadingTime = 10,
                    FeaturedImage = "https://example.com/image.jpg",
                    LastEditedDate = DateTime.Now,
                    MetaTitle = "EF Core TPT",
                    MetaKeywords = "EF Core, Inheritance, TPT",
                    MetaDescription = "Learn about TPT inheritance in EF Core  
with examples."
                };
                // Create and seed Video content
                var video = new Video
                {
                    Title = "Learn EF Core with Videos",
                    Author = "Rakesh Kumar",
                    PublishedDate = DateTime.Now,
                    ContentType = ContentType.Video,
                    Status = ContentStatus.Published,
                    VideoUrl = "http://example.com/learn-efcore.mp4",
                    ThumbnailUrl = "https://example.com/thumbnail.jpg",
                    Duration = 3600,
                    Resolution = "1080p",
                    HasSubtitles = true,
                    Subtitles = "http://example.com/subtitles.srt",
                    MetaKeywords = "EF Core, Video, Learning",
                };
            }
        }
    }
}
```

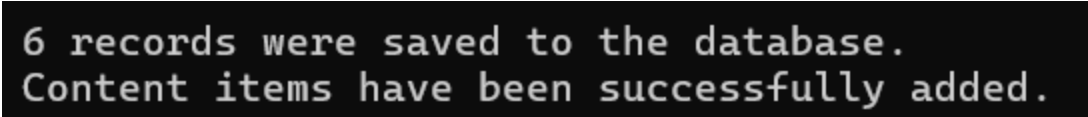
```

        MetaDescription = "Learn EF Core through comprehensive
video tutorials."
    };
    // Create and seed Image content
    var image = new Image
    {
        Title = "EF Core Infographic",
        Author = "Hina Sharma",
        PublishedDate = DateTime.Now,
        ContentType = ContentType.Image,
        Status = ContentStatus.Published,
        Caption = "EF Core Architecture Diagram",
        ImageUrl = "http://example.com/efcore-infographic.jpg",
        AltText = "EF Core Infographic",
        Dimensions = "1920x1080",
        Photographer = "Hina Sharma"
    };
    // Add the new content to the context
    context.Articles.Add(article);
    context.Videos.Add(video);
    context.Images.Add(image);
    // Save the changes to the database
    int recordsAdded = context.SaveChanges();
    // Output the result
    Console.WriteLine($"{recordsAdded} records were saved to the
database.");

    // Confirm insertion
    Console.WriteLine("Content items have been successfully
added.");
    }
    }
}

```

We create instances of Articles, Videos, and Images, each representing a specific type of content. The EFCoreDbContext is used to manage database interactions. We add these content types to their corresponding DbSet and call SaveChanges to insert them into the database. The total number of records added is displayed on the output screen, as shown in the image below.



```

6 records were saved to the database.
Content items have been successfully added.

```

Now, if you verify the database, then you should see the data in the respected database tables as shown in the below image:

SELECT * FROM [CMSDB].[dbo].[Contents]
SELECT * FROM [CMSDB].[dbo].[Articles]
SELECT * FROM [CMSDB].[dbo].[Videos]
SELECT * FROM [CMSDB].[dbo].[Images]

Results
Messages

ContentId	Title	Author	PublishedDate	ContentType	Status
1	Understanding EF Co...	Pranaya Rout	2024-09-24 10:35:54.0039212	Article	Published
2	EF Core Infographic	Hina Sharma	2024-09-24 10:35:54.0048921	Image	Published
3	Learn EF Core with V...	Rakesh Kumar	2024-09-24 10:35:54.0047957	Video	Published

ContentId	Content	Summary	ReadingTime	FeaturedImage	LastEditedDate	MetaKeywor...	MetaDescription	MetaTitle
1	This is a comprehensi...	EF Core TPT ...	10	https://exampl...	2024-09-24 10:...	EF Core, In...	Learn about TP...	EF Core TPT

ContentId	VideoUrl	Duration	ThumbnailUrl	Resolution	HasSubtitles	Subtitles	MetaKeywords	MetaDescription
3	http://exampl...	3600	https://exampl...	1080p	1	http://examp...	EF Core, Video...	Learn EF Core through compr...

ContentId	ImageUrl	Caption	AltText	Dimensions	Photographer
2	http://example.com/efcore-...	EF Core Architec...	EF Core Info...	1920x1080	Hina Sharma

Example of Read Operations Using TPT Inheritance in EF Core

When querying the base entity (Content), EF Core combines data from the base table (Contents) and the derived tables (Articles, Videos, Images) using joins. It retrieves the relevant records from their respective tables for specific queries like Articles or Videos. Now, let's retrieve the inserted data for each content type from the database and display it in the console. For a better understanding, please modify the Program class as follows:

```

using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            using (var context = new EFCoreDbContext())
            {
                // The Contents DbSet will returns records from the base
Contents table.
                // Since TPT creates separate tables for each derived class,
                // this query will hit multiple tables(Joining with Articles,
Videos, Images)
                // But we can only access the properties which are available
in base Content type
                var contents = context.Contents.ToList();
                //We loop through the contents and display the common
properties
                //such as ContentId, Title, ContentType, Author, and
PublishedDate.
                Console.WriteLine("----- List of All Content -----");
                foreach (var content in contents)
                {
                    Console.WriteLine($"Content ID: {content.ContentId},
Title: {content.Title}, Type: {content.ContentType}, Author:

```


When we update an entity, EF Core automatically updates both the base table and the derived table(s). For example, if you update an Article, common properties like Title and Author are updated in the Contents table, while properties like Summary and ReadingTime are updated in the Articles table. For a better understanding, please modify the Program class as follows:

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            using (var context = new EFCoreDbContext())
            {
                // Fetch an existing article by its primary key (ContentId)
                var article = context.Articles.FirstOrDefault(a =>
a.ContentId == 1);
                if (article != null)
                {
                    // Update the article's properties
                    article.Title = "Updated: Understanding EF Core TPT
Inheritance";
                    article.Summary = "Updated Summary for EF Core TPT
Inheritance";
                    article.ReadingTime = 12;
                    article.LastEditedDate = DateTime.Now;
                    // Save the changes to the database
                    context.SaveChanges();
                    // Output result
                    Console.WriteLine($"Article (ID: {article.ContentId}) has
been updated.");
                }
                else
                {
                    Console.WriteLine("Article not found.");
                }
                // Fetch an existing video by its primary key (ContentId)
                var video = context.Videos.FirstOrDefault(v => v.ContentId ==
3);
                if (video != null)
                {
                    // Update the video's properties
                    video.Title = "Updated: Learn EF Core with Videos";
                    video.Duration = 4500; // Updated to 75 minutes
                    video.HasSubtitles = false; // Removing subtitles
                    // Save the changes to the database
                    context.SaveChanges();
                    // Output result
                    Console.WriteLine($"Video (ID: {video.ContentId}) has
been updated.");
                }
            }
        }
    }
}
```

```

else
{
    Console.WriteLine("Video not found.");
}
// Fetch an existing image by its primary key (ContentId)
var image = context.Images.FirstOrDefault(i => i.ContentId ==
2);
if (image != null)
{
    // Update the image's properties
    image.AltText = "Updated EF Core Infographic Alt Text";
    image.Photographer = "Updated: Alice Johnson";
    // Save the changes to the database
    context.SaveChanges();
    // Output result
    Console.WriteLine($"Image (ID: {image.ContentId}) has
been updated.");
}
else
{
    Console.WriteLine("Image not found.");
}
}
}
}
}
}

```

Code Explanation:

- In the above example, we use the `FirstOrDefault` method to retrieve an existing Article, Video, or Image by their `ContentId`. The TPT inheritance ensures that EF Core knows how to join the base and derived tables to fetch the complete entity.
- Once the entity is fetched, we update its properties. In the case of Article, we update the Title, Summary, and `LastEditedDate`. For Video, we update the Duration and `HasSubtitles` property. Similarly, for Image, we modify the `AltText` and `Photographer`.
- After modifying the properties, we call `SaveChanges()` to persist the updates. EF Core handles the updates across the base (Contents) and the derived tables (Articles, Videos, Images).

For each updated entity, we display a message confirming the successful update. So, when you run the above code, you should see the following output:

```

Article (ID: 1) has been updated.
Video (ID: 3) has been updated.
Image (ID: 2) has been updated.

```

Example of Delete Operations Using TPT Inheritance in EF Core

When an entity (e.g., Article, Video, Image) is deleted, the base entity in the Contents table is removed, and the associated record in the derived table (e.g., Articles, Videos, Images) is also deleted. If a base entity is deleted, its corresponding record in the derived table is also deleted, maintaining database consistency. For a better understanding, please modify the Program class as follows:


```
context.SaveChanges();  
// Output result  
Console.WriteLine($"Image (ID: {image.ContentId}) has  
been deleted.");  
}  
else  
{  
    Console.WriteLine("Image not found.");  
}  
}  
}  
}
```

Code Explanation:

- We fetch the Article, Video, and Image entities by their ContentId. Once the entities are fetched, we call the Remove() method on the corresponding DbSet to mark the entity for deletion.
- When we call SaveChanges(), EF Core generates the necessary SQL to delete the record from both the base Contents table and the derived table (Articles, Videos, or Images), ensuring that the TPT inheritance structure is respected.

We output a message confirming that the entity has been successfully deleted. So, when you run the above code, you should see the following output:

```
Article (ID: 1) has been deleted.
Video (ID: 3) has been deleted.
Image (ID: 2) has been deleted.
```

Note: If you check the database tables, you will see that the data from both the base and derived tables has been deleted.

Advantages of Table Per Type (TPT) in Entity Framework Core

- **Normalized Database Schema:** TPT promotes a normalized design by separating common and specific properties into different tables, reducing redundancy.
- **No Null Columns:** Each table contains only the properties relevant to that entity type, eliminating null columns.
- **Data Integrity:** It is easier to enforce database constraints like NOT NULL on derived type properties.
- **Logical Separation:** Clear separation of data for different entity types, which can improve data organization and integrity.

Drawbacks of Table Per Type (TPT) in Entity Framework Core

- **Performance Overhead:** Retrieving derived entities requires joins between tables, which can degrade performance, especially with large datasets or deep inheritance hierarchies.
- **Complexity in Queries:** The generated SQL queries are more complex due to the joins, making debugging and optimization more challenging.
- **Increased Maintenance:** Managing multiple tables can increase the complexity of database maintenance tasks.

Table Per Type (TPT) Inheritance provides a way to map inheritance hierarchies in Entity Framework Core without introducing null columns in the database schema. While it offers advantages regarding data integrity and logical

separation, it comes with performance trade-offs due to the necessity of joining multiple tables. It's best suited for applications where data integrity is a priority over query performance.

In the next article, I will discuss [Table Per Concrete Type \(TPC\) Inheritance in Entity Framework Core](#) with Examples. In this article, I explain Table Per Type (TPT) Inheritance in Entity Framework Core with Examples. I hope you enjoy this TPT Inheritance in EF Core article.